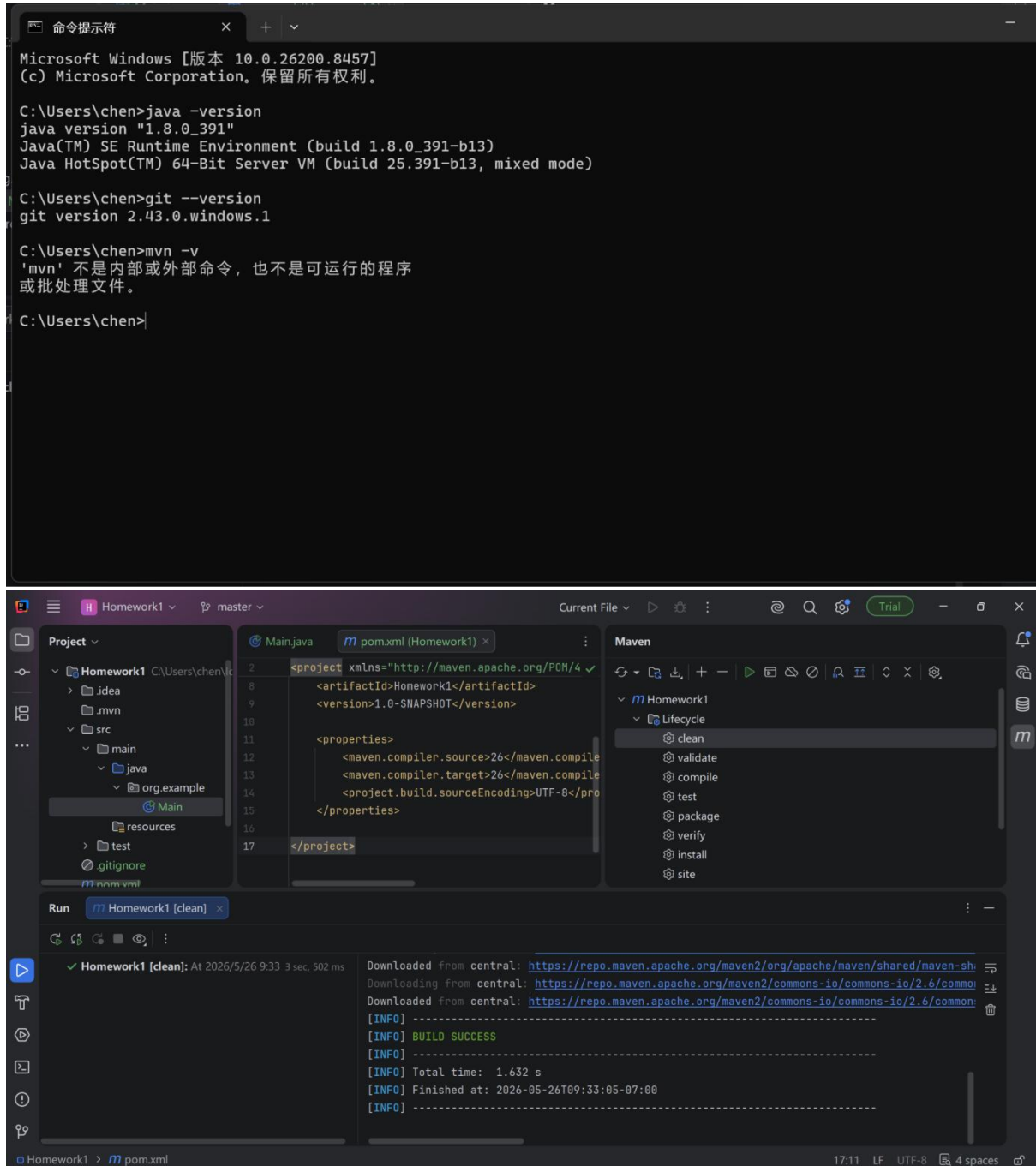
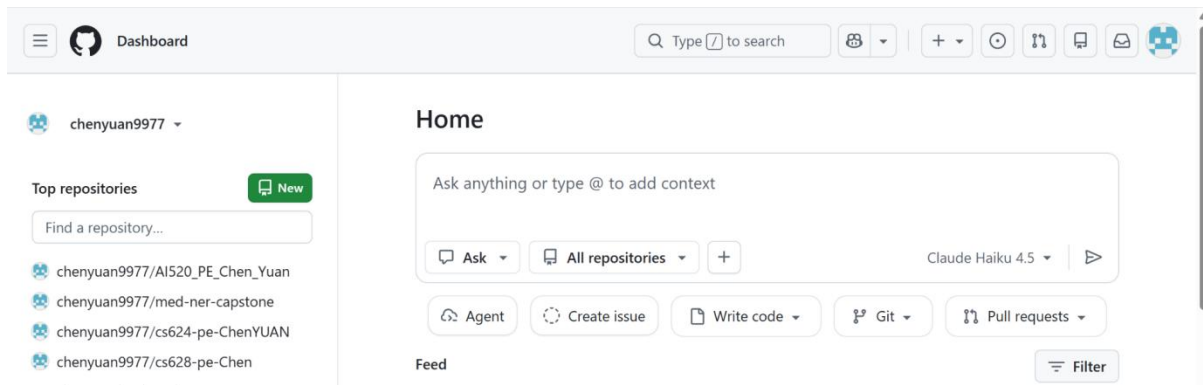


Homework1

Install git, github, maven, intellij, Java (11/17)
provide screenshot to prove install successfully





1. Question List (write necessary code to answer the following questions)

- a. List vs Set
- b. LinkedList vs ArrayList
- c. What is Map Interface
- d. How does HashMap work
- e. What is hash collision
- f. What is Collections used for
- g. What is immutable class
- h. Hashtable vs HashMap vs ConcurrentHashMap
- i. String vs StringBuilder vs StringBuffer
- j. why we need to override the hashCode and equals method at the same time
- k. play around the common data structure apis (map, set, queue, list), write some practice codes

a. List vs Set

A List is an ordered collection that allows duplicate elements and maintains their insertion order. A Set is an unordered collection that prohibits duplicate elements, ensuring every element is completely unique.

b. LinkedList vs ArrayList

ArrayList is backed by a dynamic array, making it highly efficient for retrieval ($O(1)$ random access) but slow for structural modifications. LinkedList is backed by a doubly-linked list, making it efficient for structural mutations ($O(1)$ insertions/deletions at pointers) but slow for indexing.

c. What is Map Interface

The Map interface models a mathematical function that maps unique keys to values. Because it holds key-value pairs rather than single elements and does not implement the Iterable interface, it does not belong to the standard Collection hierarchy.

d. How does HashMap work

HashMap operates on the principle of hashing using an internal array of buckets. It computes the key's hashCode() to route the pair to a specific array index, then uses equals() to pinpoint or append the exact key inside that bucket.

e. What is Hash Collision

A hash collision occurs when two completely different keys produce identical hash values, mapping them to the exact same bucket array index. Java resolves this by chaining entries in a linked list, which upgrades into a Red-Black Tree if a bucket's list grows beyond 8 elements to preserve search speed.

f. What is Collections used for

Collections (with an 's') is an uninstantiable utility class comprised strictly of static polymorphic algorithms. It is used to perform operations directly on data structures, such as sorting, searching, shuffling, reversing, or wrapping them into synchronized containers.

g. What is Immutable Class

An immutable class is a class design whose instances cannot alter their state or values after instantiation (like String). It is constructed by declaring the class as final to stop inheritance, making all fields private final, and exposing no state-changing setter methods.

h. HashTable vs HashMap vs ConcurrentHashMap

HashMap is asynchronous, fast, not thread-safe, and allows null keys/values. HashTable is a legacy thread-safe implementation that is highly inefficient because it locks the entire collection for every operation. ConcurrentHashMap is a modern thread-safe implementation that uses fine-grained locking and CAS concurrent operations to lock only the specific bucket nodes being mutated.

i. String vs StringBuilder vs StringBuffer

String is immutable, meaning any modification generates a brand new object in memory and lowers efficiency during heavy concatenation. StringBuilder and StringBuffer are mutable and modify strings in-place; StringBuilder is faster but not thread-safe, while StringBuffer is thread-safe due to synchronized methods.

j. Why we need to override hashCode and equals at the same time

The core Java contract dictates that if two objects are equal according to equals(), they must share the identical hashCode(). If you fail to override both, logically duplicate objects will generate different hashes, causing them to completely bypass uniqueness checks or get lost inside hash-based structures like HashMap or HashSet.